



Università di Catania

UNIVERSITÀ DEGLI STUDI DI CATANIA
DIPARTIMENTO DI MATEMATICA E INFORMATICA
CORSO DI LAUREA MAGISTRALE IN INFORMATICA.

Relazione Progetto *Semantic Web*

CyberSecOnto: Automazione dell'Analisi del Rischio Informatico tramite Tecnologie Semantiche

Martina Brancaforte
matricola 1000015074

Prof. Marianna Nicolosi Asmundo

Anno Accademico 2025/2026

Indice dei contenuti

Indice dei contenuti	1
1 Introduzione e Obiettivi	2
1.1 La sicurezza informatica moderna	2
1.2 Obiettivi del progetto	2
2 Modellazione Concettuale	3
2.1 Gerarchia delle Classi	3
2.1.1 Assiomi di Disgiunzione	3
2.2 Proprietà	3
2.2.1 Object Properties	4
2.2.2 Data Properties	4
2.3 Restrizioni Locali	4
2.3.1 Restrizione di Cardinalità (Exactly)	4
2.3.2 Restrizione Universale (Only)	5
2.3.3 Restrizione Esistenziale (Some)	5
3 Reasoning e Inferenza	6
3.1 Regole SWRL (Semantic Web Rule Language)	6
3.1.1 Regola 1: Classificazione Vulnerabilità Critica	6
3.1.2 Regola 2: Propagazione del Rischio (High Risk)	6
3.2 Inferenza tramite Classi Equivalenti (OWL)	7
4 Implementazione Software e Automazione	8
4.1 Ambiente di Sviluppo (Visual Studio Code)	8
4.2 Integrazione Python con Owlready2	8
4.3 Script di Generazione Procedurale	8
4.4 Motore Inferenziale Ibrido	9
5 Interrogazione (SPARQL)	10
5.1 Query: Report Sistemi Critici	10
5.2 Query: Sistemi Vulnerabili (Inferenza OWL)	10
6 Conclusioni e Sviluppi Futuri	12

1.1 La sicurezza informatica moderna

La sicurezza informatica moderna (Cybersecurity) è caratterizzata da una vasta eterogeneità di informazioni: asset hardware, vulnerabilità software, tipologie di attacco e contromisure. La gestione di queste informazioni tramite database relazionali tradizionali risulta spesso rigida e incapace di evidenziare relazioni implicite, come l'impatto a catena di un guasto o il livello di rischio derivato. In questo progetto è stata sviluppata CyberSecOnto, un'ontologia in linguaggio OWL (Web Ontology Language) progettata per modellare lo scenario della sicurezza aziendale.

1.2 Obiettivi del progetto

L'obiettivo principale non è la semplice archiviazione dei dati, ma l'automazione del processo di analisi del rischio. Attraverso l'uso di tecnologie del Web Semantico, il sistema è in grado di:

- Modellare l'infrastruttura IT e le dipendenze tra i sistemi.
- Inferire automaticamente la criticità delle vulnerabilità basandosi su punteggi standard (CVSS).
- Classificare i sistemi a rischio senza intervento umano, sfruttando sia regole SWRL che inferenze OWL standard.
- Validare la consistenza dei dati tramite restrizioni logiche.

La struttura terminologica (T-Box) è stata definita utilizzando l'editor Protégé. Il dominio è stato modellato attraverso una gerarchia di classi, proprietà semantiche e restrizioni logiche.

2.1 Gerarchia delle Classi

Le entità principali seguono una tassonomia che riflette le componenti di un ecosistema cyber:

- **System:** La classe radice per gli asset hardware. Include le sottoclassi Server e Workstation.
- **Vulnerability:** Rappresenta le debolezze software (es. CVE). Include la sottoclasse SoftwareVulnerability.
- **Attack:** Rappresenta le tipologie di attacco (es. Malware, Ransomware).
- **Mitigation:** Rappresenta le contromisure difensive (es. Firewall, Patch).
- **ThreatActor:** Gli agenti di minaccia (es. Hacker, NationState).

2.1.1 Assiomi di Disgiunzione

Per garantire la coerenza logica del modello, è stata applicata la disgiunzione (Disjoint Classes) tra le classi di alto livello. Ad esempio, un individuo non può appartenere contemporaneamente alla classe System e alla classe AttackType.

2.2 Proprietà

Le relazioni semantiche definiscono le interazioni tra gli individui.

2.2.1 Object Properties

- **hasVulnerability** (Domain: System $\rightarrow$ Range: Vulnerability): Associa un asset alla sua debolezza.
- **exploits** (Domain: AttackType $\rightarrow$ Range: Vulnerability): Indica quale falla viene sfruttata da un attacco.
- **mitigates** (Domain: Mitigation $\rightarrow$ Range: Vulnerability): Indica quale difesa risolve il problema.
- **dependsOn** (Domain: System $\rightarrow$ Range: System): Modella la dipendenza operativa tra macchine.

La Transitività La proprietà `dependsOn` è stata definita come Transitiva. Se il Sistema A dipende da B, e B dipende da C, il Reasoner inferisce automaticamente che A dipende da C. Questo è fondamentale per l'analisi dell'impatto a catena in infrastrutture complesse.

2.2.2 Data Properties

- **hasCVSSScore** (Domain: Vulnerability $\rightarrow$ Range: xsd:decimal):

Assegna il punteggio numerico di gravità (Common Vulnerability Scoring System). È stato scelto il tipo decimal per garantire la compatibilità con i calcoli di precisione.

2.3 Restrizioni Locali

Per aumentare la robustezza e l'espressività del modello, sono state implementate tre tipologie di restrizioni OWL.

2.3.1 Restrizione di Cardinalità (Exactly)

Per garantire l'integrità dei dati, è stato imposto che ogni vulnerabilità debba avere un unico punteggio di gravità.

- **Classe:** Vulnerability
- **Assioma:** `hasCVSSScore exactly 1 xsd:decimal`

Inoltre, la proprietà `hasCVSSScore` è stata resa Funzionale, generando un'inconsistenza (errore) qualora a un individuo vengano assegnati due punteggi distinti.

2.3.2 Restrizione Universale (Only)

Per vincolare il dominio degli attacchi, è stato stabilito che un attacco può sfruttare soltanto vulnerabilità.

- **Classe:** AttackType
- **Assioma:** exploits only Vulnerability
Questo previene errori di modellazione, come il collegamento errato di un attacco direttamente a un server.

2.3.3 Restrizione Esistenziale (Some)

È stata definita una classe speciale per la classificazione automatica.

- **Classe:** VulnerableSystem
- **Assioma:** System and (hasVulnerability some Vulnerability)

Il valore aggiunto del progetto risiede nella capacità di dedurre nuova conoscenza non esplicitamente asserita. Sono state utilizzate due tecniche complementari: le regole SWRL e l'inferenza standard OWL.

3.1 Regole SWRL (Semantic Web Rule Language)

Le regole SWRL sono state utilizzate per calcoli che coinvolgono valori numerici e propagazione condizionale.

3.1.1 Regola 1: Classificazione Vulnerabilità Critica

Questa regola valuta il punteggio CVSS. Se supera la soglia di 8.0, la vulnerabilità viene riclassificata.

```
1 PREFIX : <http://www.semanticweb.org/salvleanza/ontologies/  
  cybersOnto#>  
2 SELECT ?system  
3 WHERE {  
4     ?system a :VulnerableSystem .  
5 }
```

Figura 3.1: Regola CriticalVulnerability

3.1.2 Regola 2: Propagazione del Rischio (High Risk)

Questa regola propaga la criticità dalla vulnerabilità al sistema che la ospita.

```
1 PREFIX : <http://www.semanticweb.org/salvleanza/ontologies/  
  cybersOnto#>  
2 SELECT ?system  
3 WHERE {  
4     ?system a :VulnerableSystem .  
5 }
```

Figura 3.2: Regola HighRiskSystem

3.2 Inferenza tramite Classi Equivalenti (OWL)

Per rispondere alla necessità di classificare i sistemi in base alla presenza di vulnerabilità (indipendentemente dalla gravità), è stata utilizzata una Classe Equivalente definita tramite restrizione esistenziale. La classe `VulnerableSystem` è stata definita come:

```
System and (hasVulnerability some Vulnerability)
```

Il ***Reasoner*** (Pellet) analizza gli individui definiti come generici `System`: se rileva la presenza di almeno una relazione **hasVulnerability**, inferisce automaticamente l'appartenenza alla classe **VulnerableSystem**.

4 Implementazione Software e Automazione

Per gestire la complessità e garantire la scalabilità del progetto, lo sviluppo non si è limitato all'uso dell'editor grafico Protégé. È stata implementata una pipeline di automazione completa per simulare scenari Enterprise e superare le limitazioni computazionali dei reasoner standard.

4.1 Ambiente di Sviluppo (Visual Studio Code)

L'intero ciclo di vita del software è stato gestito utilizzando Visual Studio Code. Il progetto è stato strutturato in un repository GitHub che contiene:

- I file sorgente dell'ontologia (*Untitled.rdf*, *cyberseconto.owl*).
- Gli script Python di automazione (*popolamento.py*, *apply_swrl_rules.py*).
- Gli script di test e validazione (*test_sparql.py*, *debug_onto.py*).
- L'uso di un sistema di controllo versione ha permesso di tracciare l'evoluzione del modello semantico e di separare la logica applicativa dai dati ontologici.

4.2 Integrazione Python con Owlready2

L'interazione programmatica con l'ontologia è stata realizzata tramite la libreria Owlready2, che consente di mappare le classi OWL in classi Python. Questo ha permesso di manipolare individui e proprietà come oggetti nativi del linguaggio.

4.3 Script di Generazione Procedurale

Lo script *popolamento.py* funge da simulatore di infrastruttura. Invece di popolare manualmente, lo script:

4.4 Motore Inferenziale Ibrido

Data la complessità computazionale di alcuni Reasoner nel gestire atomi SWRL matematici su dataset estesi, è stato implementato un motore di inferenza esterno tramite lo script `apply_swrl_rules.py`.

Questo script analizza i dati, applica la logica delle regole SWRL e inietta i risultati direttamente nell'ontologia. Questo approccio permette di generare un file finale denominato `cyberseconto_inferred.owl`, che rappresenta la versione definitiva, più densa e completa dell'intero progetto.

Tale file racchiude la totalità della base di conoscenza, consolidando in un unico ambiente sia i dati popolati che le deduzioni logiche estratte, rendendole persistenti e indipendenti dal software di visualizzazione utilizzato.

Le query SPARQL presentate in questo capitolo sono state il frutto di un processo iterativo di progettazione. Inizialmente, le interrogazioni sono state pensate e generate programmaticamente all'interno degli script Python per facilitare l'estrazione automatizzata dei dati durante la fase di sviluppo. Successivamente, tali query sono state validate e testate singolarmente nell'ambiente *Protégé* per assicurarne la correttezza semantica e la precisione dei risultati.

È importante sottolineare che nella presente relazione sono state riportate esclusivamente alcune tra le query ai fini della dimostrazione dell'efficacia del modello. L'intera suite di interrogazioni, comprese quelle utilizzate per il debugging e la manutenzione dei dati, è consultabile integralmente all'interno degli script di repository

5.1 Query: Report Sistemi Critici

Questa query estrae tutti i sistemi che il Reasoner ha classificato come "Ad Alto Rischio" e mostra il relativo punteggio di gravità, ordinandoli in modo decrescente.

```
1 PREFIX : <http://www.semanticweb.org/salvleanza/ontologies/  
  cybersOnto#>  
2 SELECT ?system  
3 WHERE {  
4   ?system a :VulnerableSystem .  
5 }
```

Figura 5.1: SPARQL Query per HighRiskSystem

5.2 Query: Sistemi Vulnerabili (Inferenza OWL)

Questa query interroga la classe definita tramite restrizione esistenziale.

```
1 PREFIX : <http://www.semanticweb.org/salvleanza/ontologies/  
    cybersOnto#>  
2 SELECT ?system  
3 WHERE {  
4     ?system a :VulnerableSystem .  
5 }
```

Figura 5.2: SPARQL Query per VulnerableSystem

6 Conclusioni e Sviluppi Futuri

Il progetto **CyberSecOnto** ha ampiamente dimostrato l'efficacia e il valore aggiunto delle tecnologie del Web Semantico nel dominio della sicurezza informatica[cite: 355]. Rispetto ai tradizionali database relazionali, caratterizzati da una forte rigidità strutturale [cite: 4, 214], l'approccio basato sui grafi semantici ha permesso di mappare ed estrarre automaticamente relazioni implicite e complesse[cite: 9, 214]. L'uso di proprietà transitive come **dependsOn** si è rivelato fondamentale per l'analisi proattiva dell'impatto a catena all'interno di infrastrutture IT complesse[cite: 29, 32, 248].

L'architettura implementata ha evidenziato una perfetta complementarità tra i diversi strumenti tecnologici adottati[cite: 11]:

- **OWL**: ha garantito la coerenza della T-Box, la validazione dei dati tramite proprietà funzionali e restrizioni di cardinalità (**exactly 1** per il punteggio CVSS) e la classificazione automatica concettuale mediante restrizioni universali ed esistenziali.
- **SWRL**: ha permesso di introdurre logiche condizionali e confronti matematici sui valori numerici del CVSS, superando i limiti espressivi dello standard OWL.
- **Python ed Owlready2**: hanno automatizzato il popolamento procedurale di uno scenario Enterprise realistico, realizzando un motore inferenziale ibrido capace di aggirare i limiti computazionali dei reasoner grafici tradizionali e di materializzare le inferenze in modo persistente nel file `cyberseconto_inferred.owl`.

La validazione finale tramite query **SPARQL** ha confermato la correttezza delle deduzioni logiche, dimostrando come il sistema sia in grado di generare nuova conoscenza e identificare autonomamente gli asset prioritari a rischio (**HighRiskSystem**) senza la necessità di query SQL predefinite o interventi manuali.

Come ****sviluppi futuri****, il framework **CyberSecOnto** potrebbe essere esteso per integrare dinamicamente feed di minacce in tempo reale (come il database MITRE ATT&CK o i cataloghi CISA KEV) o per interfacciarsi con sistemi di monitoraggio aziendali (SIEM),

automatizzando non solo la scoperta del rischio ma anche la raccomandazione in tempo reale delle contromisure (**Mitigation**) più idonee.

I sorgenti completi del progetto, le ontologie e gli script di automazione sono liberamente consultabili e riproducibili presso la repository GitHub dedicata: <https://github.com/Marty14/Semantic-Web>.